Create and inspect DataFrames

Understand rows, columns, indexes and common first checks

1 Start in your VS Code notebook

Save a .ipynb notebook in ML4EngApp. Select the MachineLearning4EngApp kernel and run Code cells with Shift+Enter. If needed, run %pip install pandas, restart the kernel and Run All.

A DataFrame is a two-dimensional table. Each column is a Series and can have a different data type. The index holds row labels; it is separate from any ID column. These examples use a small table with one deliberately missing value.

```
import pandas as pd
df = pd.DataFrame({
    "id": ["A", "B", "C", "D"],
    "rul": [80.0, 30.0, None, 10.0],
    "state": ["healthy", "worn", "worn", "worn"]
})
base = df.copy()  # keep original teaching data
print(df)
```

Output

```
  id   rul    state
0  A  80.0  healthy
1  B  30.0     worn
2  C   NaN     worn
3  D  10.0     worn
```

Dictionary keys become column names. Each list supplies one column and must have the same length. None becomes a missing numeric value (NaN) here. RUL represents hours remaining.

2 Another way to create a table

```
rows = [["A", 80], ["B", 30]]
small = pd.DataFrame(rows, columns=["id", "rul"])
print(small)
```

Output

```
  id  rul
0  A   80
1  B   30
```

A list of rows needs column names. You can also construct a DataFrame from a list of dictionaries or a two-dimensional NumPy array.

3 First and last rows

```
print(df.head(2))  # first two rows
print(df.tail(1))  # last row
```

Output

```
  id   rul    state
0  A  80.0  healthy
1  B  30.0     worn
  id   rul state
3  D  10.0  worn
```

4 Shape, columns and data types

```
print(df.shape)  # (rows, columns)
print(df.columns.tolist())
print(df.index.tolist())
print(df.dtypes)
```

Output

```
(4, 3)
['id', 'rul', 'state']
[0, 1, 2, 3]
id        object
rul      float64
state     object
dtype: object
```

shape, columns, index and dtypes are attributes: no parentheses. head(), info() and describe() are methods: they need (). Text dtypes may display as object, str or string depending on your pandas version.

5 A compact summary

```
print(df["rul"].describe().round(1))
```

describe() reports count, mean, standard deviation, minimum, quartiles and maximum for numeric data. Missing RUL is excluded, so count is 3 rather than 4. df.describe(include="all") also summarises non-numeric columns.

```
df.info()  # types and non-null counts
```

info() prints a technical summary. It does not return a summary table: print(df.info()) adds an unhelpful None after its output.

6 Load a CSV DataFrame

```
loaded = pd.read_csv("vibration_rul_dataset.csv")
print(loaded.shape)
print(loaded[["machine_id", "rul_hours"]].head(2))
```

Output

```
(738, 1030)
   machine_id  rul_hours
0           0  210.00764
1           0  204.75745
```

Keep the CSV beside your notebook, or supply its full path. loaded is separate from our small teaching df.
For IDs with leading zeros, if the CSV has an 'id' column, use pd.read_csv(path, dtype={'id': 'string'}). Other options: sep=';', usecols=[...] and na_values=['unknown'].

On pages 2 to 5, run df = base.copy() first to reset the small example. This keeps the examples independent of earlier edits.

Select, slice and filter

Use column names, row labels, positions and Boolean conditions

```
df = base.copy()  # reset the four-row example
```

1 One column versus a table

```
print(type(df["rul"]).__name__)
print(type(df[["rul"]]).__name__)
print(df[["id", "rul"]])
```

Output

```
Series
DataFrame
  id   rul
0  A  80.0
1  B  30.0
2  C   NaN
3  D  10.0
```

df["rul"] returns a Series. Double brackets select a list of column names and return a DataFrame, even when there is only one column. Column names are case sensitive.

2 .iloc selects by position

```
print(df.iloc[0, 1])    # first row, second column
print(df.iloc[:2, :2])  # first 2 rows and columns
print(df.iloc[-1, 0])   # last row, first column
```

Output

```
80.0
  id   rul
0  A  80.0
1  B  30.0
D
```

Positions begin at zero. With .iloc[start:stop], stop is excluded. A colon alone means all rows or columns. df.iloc[0] returns one row as a Series; df.iloc[[0]] keeps a one-row DataFrame.

3 .loc selects by label

```
named = df.set_index("id")
print(named.loc["B", "rul"])
print(named.loc["A":"C", ["rul"]])
```

Output

```
30.0
     rul
id
A   80.0
B   30.0
C    NaN
```

The index is now A, B, C, D. .loc[rows, columns] uses these labels and column names. Unlike .iloc, this ordered label slice includes C. .loc[0] means label 0, not necessarily the first row.

4 Filter rows with a condition

```
mask = df["rul"] <= 40
print(df.loc[mask, ["id", "rul"]])
```

Output

```
  id   rul
1  B  30.0
3  D  10.0
```

The comparison creates one Boolean per row. .loc keeps rows marked True. A missing RUL does not pass this numeric condition. Do not compare missing values with == None or == NaN; use isna().

5 Combine conditions

```
selected = df.loc[
    (df["state"] == "worn") &
    (df["rul"] <= 40)
 ]
print(selected["id"].tolist())
in_list = df["id"].isin(["A", "D"])
print(df.loc[in_list, "id"].tolist())
```

Output

```
['B', 'D']
['A', 'D']
```

Use & for and, | for or and ~ for not; put parentheses around each comparison. Python and/or cannot combine whole Series. isin() tests membership in a collection.

6 Edit selected cells safely

```
df.loc[df["id"] == "B", "rul"] = 25
print(df.loc[1, "id"], df.loc[1, "rul"])
```

Output

```
B 25.0
```

Use one .loc[...] assignment to edit the original table. Avoid df[mask]["rul"] = 25: chained selection is not a reliable way to update df. To work on a separate subset, use subset = df.loc[mask].copy().

7 Single cells and resetting indexes

```
print(df.at[0, "id"])  # one label-based cell
print(df.iat[0, 0])    # one positional cell
print(named.reset_index().columns.tolist())
```

Output

```
A
A
['id', 'rul', 'state']
```

set_index moves a column into the row index; reset_index moves it back. After filtering, reset_index(drop=True) creates fresh 0-based labels without retaining the old index as a column.

Missing values and cleaning

Detect gaps, choose a treatment and check your assumptions

```
df = base.copy()
```

1 Find and count missing values

```
print(df.isna().sum())  # count per column
print(df["rul"].isna().tolist())
print(df.loc[df["rul"].isna(), "id"].tolist())
print(df["rul"].notna().sum())
```

Output

```
id       0
rul      1
state    0
dtype: int64
[False, False, True, False]
['C']
3
```

isna() returns a Boolean mask; summing True values counts gaps. notna() is the inverse. df.isna().any(axis=1) flags rows with any missing field. Numeric NaN, None and pd.NA can represent missing data, depending on dtype.

2 Drop only relevant missing rows

```
clean = df.dropna(subset=["rul"])
print(clean["id"].tolist())
print(len(df), len(clean))
```

Output

```
['A', 'B', 'D']
4 3
```

dropna(subset=[...]) removes rows missing the specified columns. df.dropna() removes a row if any column is missing by default. Both return a new table here; the original df is unchanged.

3 Fill values only with a reason

```
filled = df.copy()
filled["rul"] = filled["rul"].fillna(
    filled["rul"].median()
 )
print(filled["rul"].tolist())
```

Output

```
[80.0, 30.0, 30.0, 10.0]
```

The median is 30 in this example. Imputation is a modelling choice, not a way to reveal the unknown true value. Do not fill missing RUL with zero: zero means failure. Fit imputation rules on training data only when building a model.

Missing-value checks do not detect infinity. For numeric arrays, np.isfinite(...) detects NaN and positive or negative infinity.

4 Convert types and detect bad text

```
raw = pd.Series(["80", "unknown", "10"])
numeric = pd.to_numeric(raw, errors="coerce")
print(numeric.tolist())
print(numeric.isna().sum())
```

Output

```
[80.0, nan, 10.0]
1
```

errors="coerce" converts invalid numeric text to missing values. Inspect those rows rather than silently discarding them. astype() converts valid data to a chosen type and normally raises an error for invalid input.

```
df["id"] = df["id"].astype("string")
dates = pd.to_datetime(
    ["2026-10-01", "bad"], errors="coerce"
 )
print(dates.isna().tolist())
```

Output

```
[False, True]
```

pd.to_datetime creates date/time values. Invalid dates become NaT here. Keep identifiers as text when leading zeros matter.

5 Duplicates are a separate issue

```
twice = pd.concat([df, df.iloc[[0]]],
                  ignore_index=True)
print(twice.duplicated().sum())
unique = twice.drop_duplicates()
print(len(unique))
```

Output

```
1
4
```

duplicated() flags later identical rows by default. subset=["id"] checks repeated IDs instead. Repeated measurements can be legitimate: decide what uniquely identifies a record before removing duplicates.

6 Cleaning checks

```
print(df["id"].is_unique)
print(df["rul"].dropna().ge(0).all())
```

Output

```
True
True
```

is_unique checks IDs; ge(0) means >= 0. Dropping missing values before a check does not prove labels are complete: check missing counts separately. Try it: add a duplicate ID and a negative RUL, then identify both problems.

Transform, sort and summarise

Create columns, clean text and compare groups

```
df = base.copy()
```

1 Calculate a new column

```
df["rul_days"] = df["rul"] / 24
print(df["rul_days"].round(2).tolist())
```

Output

```
[3.33, 1.25, nan, 0.42]
```

Column arithmetic applies to every row. Missing values remain missing in this calculation. Most operations return a new result: assign it to a variable if you want to keep it.

2 Rename, drop and sort

```
renamed = df.rename(columns={"rul": "rul_hours"})
print(renamed.columns.tolist())
short = df.drop(columns=["rul_days"])
ordered = df.sort_values("rul", na_position="last")
print(ordered["id"].tolist())
```

Output

```
['id', 'rul_hours', 'state', 'rul_days']
['D', 'B', 'A', 'C']
```

rename changes labels; drop(columns=...) removes columns. sort_values orders by values; ascending=False reverses the order. sort_index orders by row labels. These commands do not change df unless you assign their returned tables.

3 Clean text with .str

```
text = pd.Series([" Healthy ", "WORN"])
print(text.str.strip().str.lower().tolist())
print(df["state"].str.contains("worn",
                               na=False).tolist())
```

Output

```
['healthy', 'worn']
[False, True, True, True]
```

.str exposes string operations for a Series. strip removes outer spaces; lower normalises case; contains finds matches. For literal text containing punctuation, set regex=False.

4 Map categories to codes

```
codes = {"healthy": 0, "worn": 1}
df["state_code"] = df["state"].map(codes)
print(df["state_code"].tolist())
```

Output

```
[0, 1, 1, 1]
```

map applies a lookup; unmatched keys become missing. Category codes may not be appropriate model features.

5 Common numeric summaries

```
print(df["rul"].mean())
print(df["rul"].median())
print(df["rul"].min(), df["rul"].max())
print(df["rul"].count(), len(df))
print(df["state"].value_counts())
```

Output

```
40.0
30.0
10.0 80.0
3 4
state
worn       3
healthy    1
Name: count, dtype: int64
```

count() counts non-missing values; len(df) counts rows. Numeric summaries usually skip missing values. std() uses a sample standard deviation by default (ddof=1). value_counts excludes missing categories unless dropna=False.

6 Summaries by group

```
summary = df.groupby("state")["rul"].agg(
    ["count", "mean", "min", "max"]
 )
print(summary)
```

Output

```
         count  mean   min   max
state
healthy      1  80.0  80.0  80.0
worn         2  20.0  10.0  30.0
```

groupby splits rows by category; agg calculates statistics for each group. The worn count is 2 valid RULs, but there are 3 worn rows. .size() counts rows. Missing keys are excluded unless dropna=False.

7 A simple pivot table

```
pivot = df.pivot_table(index="state",
    values="rul", aggfunc="mean")
print(pivot)
```

Output

```
          rul
state
healthy  80.0
worn     20.0
```

pivot_table reshapes grouped summaries into a table. Add columns="another_category" for a two-way summary. pd.crosstab(df["state"], other_column) gives category counts.

Try it: add machine E with RUL 50 and state healthy. Recompute the mean by state. Explain why a missing RUL should not count as zero.

Combine, export and check tables

Join related records and use a repeatable data-handling workflow

```
df = base.copy()
```

1 Concatenate compatible rows

```
new = pd.DataFrame({"id": ["E"], "rul": [50.0],
                    "state": ["healthy"]})
combined = pd.concat([df, new], ignore_index=True)
print(combined["id"].tolist())
```

Output

```
['A', 'B', 'C', 'D', 'E']
```

concat stacks tables along rows by default and matches columns by name. ignore_index=True gives fresh row labels. A column absent from one input produces missing values for those rows. Use concat rather than the removed DataFrame.append method.

2 Merge on an identifier

```
details = pd.DataFrame({"id": ["A", "B"],
    "location": ["Lab 1", "Lab 2"]})
joined = df.merge(details, on="id", how="left",
                  validate="many_to_one")
print(joined[["id", "location"]])
```

Output

```
  id location
0  A    Lab 1
1  B    Lab 2
2  C      NaN
3  D      NaN
```

A left join keeps every row in df and adds matching details. Unmatched IDs get missing locations. An inner join keeps matches only. Duplicate keys can multiply rows: validate checks the expected relationship, here one details row per ID.

3 Save and reload a CSV

```
df.to_csv("practice_dataframe.csv", index=False)
again = pd.read_csv("practice_dataframe.csv")
print(again.shape)
```

Output

```
(4, 3)
```

index=False avoids exporting extra row-index labels. CSV does not preserve every pandas dtype; check dtypes when reloading. Dates may need parse_dates=[...]. Keep an original file and save cleaned output under a new name.

For Excel, use df.to_excel("output.xlsx", index=False) and pd.read_excel(...). Install an appropriate engine such as openpyxl if required.

4 A simple processing recipe

Load data. Inspect head, shape, columns, dtypes and info. Count missing values and check duplicates. Select needed rows and columns. Convert types deliberately. Clean and calculate features. Summarise, verify and export.

5 Common mistakes to avoid

KeyError: print(df.columns.tolist()) and check spelling or extra spaces. FileNotFoundError: check the path and print(Path.cwd()). NameError: run the cell defining df. Forgetting (): df.head is a method object; df.head() runs it.

A row index is not necessarily an ID or a position. Use .iloc for positions and .loc for labels. After filtering or sorting, inspect the index before updating records.

Series arithmetic aligns by index labels. That is useful, but two equally long Series with different indexes may produce unexpected missing values. Check labels before combining columns.

6 Prefer columns to row loops

```
days = df["rul"] / 24  # a vectorised calculation
rows = df[["id", "rul"]].itertuples(index=False)
for row in rows:
    print(row.id, row.rul)
```

Output

```
A 80.0
B 30.0
C nan
D 10.0
```

Use whole-column operations when possible. If a row loop is genuinely needed, itertuples yields named row values. Do not expect modifying those row objects to update df. Use .loc for edits.

7 Practice task

From base.copy(), select worn machines, exclude rows with missing RUL, add rul_days, sort by RUL and export the result. Predict the remaining IDs first. Then merge the location table and explain the missing locations.

Restart Kernel and Run All to check reproducibility. Output formatting may vary with pandas version. This guide covers common beginner operations, not the entire pandas API.